Assignment #2: C++ Classes

Due Date 1: Sunday, October 21, 2026, 5:00 pm EST
Due Date 2: Sunday, October 28, 2026, 5:00 pm EST

Learning objectives:

- C++ classes, constructors, destructors, and operations
- Dynamic memory allocation for C++ objects and arrays
- Invariants and Encapsulation

- **Questions 2a, and 3a are due on Due Date 1; questions 1, 2b, and 3b are due on Due Date 2.**
- On this and subsequent assignments, you will take responsibility for your own testing. This assignment is designed to get you into the habit of thinking about testing *before* you start writing your program. For each question, you will be given a compiled executable program which is a program representing a solution to each question. You should use these provided executables to help you write your test cases, as they can show you the resultant output for given inputs. If you look at the deliverables and their due dates, you will notice that there is *no* C++ code due on Due Date 1. Instead, you will be asked to submit test suites for C++ programs that you will later submit by Due Date 2.

  Test suites will be in a format compatible with that presented in class.
- Design your test suites with care; they are your primary tool for verifying the correctness of your code. Note that test suite submission `zip` files are restricted to contain a maximum of 40 tests. The size of each input `(.in)` file is also restricted to 300 bytes, and each output file `(.out)` is restricted to 1,000 bytes. This is to encourage you not to combine all of your testing in a single pair of files.
- You must use the standard C++ I/O streaming and memory management (MM) facilities on this assignment; you may **not** use C-style I/O or MM. More concretely, you may `import` the following C++ libraries (and no others!) for the current assignment: `iostream`, `fstream`, `sstream`, `iomanip`, `string`, and `utility`. MarkUs will be setup to **reject** submissions that use C-style I/O or MM, or libraries other than the ones specified above.
- We will manually check that you follow a reasonable standard of documentation and style, and to verify any assignment requirements that are not automatically enforced by MarkUs. Code to a standard that you would expect from someone else if you had to maintain their code. Further comments on coding guidelines can be found here: `https://cmsweb.utsc.utoronto.ca/cscb07f26/codingGuidelines/`
- We have provided some code and sample executables in the subdirectory `codeForStudents` under the appropriate subdirectories. **These executables have been compiled on the Mathlab server and will not run anywhere else.**
- **You may not ask public questions on Piazza about what the programs that make up the assignment are supposed to do.** A major part of this assignment involves designing test cases, and questions that ask what the programs should do in one case, or another will give away potential test cases to the rest of the class. Questions found in violation of this rule will be marked private or deleted; repeat offences could be subject to discipline.
- You may publicly discuss the coding assessment portion of the questions on Piazza as long as solutions are neither discussed nor revealed.

Question 1

1. **(DD2: 10%)**In this exercise, you will write a C++ class (implemented as a `struct`) to simulate a simple hand-held calculator. The calculator keeps track of the following state:

   - the current contents of the display (`display`);
   - the current computed result (`result`), with which the displayed value will eventually be combined;
   - the contents of the memory register (`memory`), used to emulate the M+/MR/MC functionality on calculators;
   - the pending operator (`oper`): the most recently pressed operator key (a character: +, -, *, or /), or the null character (`0`) if no operator is pending;
   - whether or not the most recent operation produced an error (`error`).

   All numeric values (`display`, `result`, and `memory`) are `int`s, and all arithmetic is `int` arithmetic. In particular, division is integer division (the quotient is truncated toward zero). Whenever two values are combined with an operator, `result` (the value computed so far) is the left operand and `display` (the number most recently typed) is the right operand. For example, when `7-2=` is entered, `result` holds `7` and `display` holds `2` at the time the subtraction is performed, so the calculator computes `7 - 2`, not `2 - 7`.

   Throughout this description, the term *memory register* refers only to the single value `memory` used by M+/MR/MC; the *state* of the calculator refers to all five values listed above.

   You will use the provided `calc.cc` file as the basis for your class. **You are not allowed to change `calc.cc`.**

   Note on M+/MR/MC: Simple hand-held calculators often give you one `int` of memory to store a value for later use. The memory register initially contains the value `0`.

   - Pressing `M+` adds the displayed value to the contents of the memory register, and stores the result in the memory register. The display is left unchanged.
   - Pressing `MR` copies the contents of the memory register to the display, replacing whatever was displayed. Digits typed afterwards are appended to the recalled value, just as with any other displayed value.
   - Pressing `MC` clears the contents of the memory register (sets it to zero).

   Note that an `M` is shown in the output whenever the memory register is non-zero (see below).`M` depends only on the current value of the register, not on whether `M+` has ever been pressed. For example, if the register holds `3` and `M+` is pressed while the display shows `-3`, the register becomes `0` and `M` no longer appears.

   You are to implement the following methods:

   - `void digit(int digit)` — adds `digit` (which must be between 0 and 9, inclusive) to the end of the displayed number. So if the display contains `-5` and `3` is typed, the display becomes `-53`.
   - `void op(char oper)` — sets the `oper` field. If the `oper` field previously contained an operator, it combines result and display with that operator (computing *result operator display*), and stores the result in `result`. If not, display is copied to result. In either case, display is cleared (reset to 0).
   - `void equals()` — combines result and display using `oper` (computing *result operator display*), and stores the result in result and display. Clears the `oper` field. If no operator is pending, `display` is copied to `result` (so the display does not change). This means that pressing `=` several times in a row has no further effect after the first press: after `2+3=`, pressing `=` again displays `5` again.
   - `void memPlus(), void memClear(), void memRecall()` — as above
   - `bool isError() const` — answers true if the error flag is set (because of division by 0).
   - `void allClear()` — resets the entire state of the calculator: display, result, and the memory register are set to 0, the operator is cleared, and the error flag is reset.

   Division by zero: If `op` or `equals` would perform a division whose divisor is 0, the division is not performed. Instead, `result` is set to 0 (and, in the case of `equals`, so is the display), and the error flag is set. In `op`, the new operator is still recorded and the display is still cleared, so the calculation continues from a result of 0.

Once set, the error flag stays set for the rest of the current calculation. Because the error flag persists until the end of the calculation, an error is always visible in the output printed by the next `=`. For example, `6/0=` prints `0 E`, and so does `6/0+=`. Likewise `6/0+3=` prints `3 E`: the division by zero sets the error flag and makes the result 0, and then `0+3` is computed with the flag still set. Pressing `=` again immediately afterwards starts no new operation, since no operator is pending, so it resets the error flag (and prints just `3`).

You are also to implement the output operator, which works as follows:

- if memory contains 0, just print out the value of display;
- if memory is non-zero, print out

  value-of-display `M:` *value-of-memory*

- if the error flag is set, append `E` (a space and an E) to the above.

We have provided the file `main.cc`, which you can use to test your `Calc` class. A sample session follows (lines in black are input, lines in blue are output):

```
456-123+456=
789
1+1=
7892
AC
1+1=
2
M+
=
2 M: 2
/0
=
0 M: 2 E
AC
=
0
```

Note the second calculation: because the display already contains `789`, the `1+1` actually appends the `1` to `789`, and requests `7891+1` (`7892`). This was done for reasons of simplicity.

Note the fourth calculation: `M+` adds the displayed `2` to the memory register but does not change the display, so the following `=` (with no pending operator) simply prints `2` along with the new memory value. For the same reason, the display still contains `2` when `/0` is entered, so the next `=` attempts `2/0`, which sets the error flag and produces `0 M: 2 E`. Finally, `AC` resets everything, including the memory register, so the last `=` prints just `0`.

Note that there are no spaces between the digits and the operators/commands. The program terminates normally when there is no more input (EOF).

We have provided a sample executable in your `cscb07f26/a2` directory, which you can use to disambiguate questions about how the calculator should work. This question has public tests that you can run on MarkUs. The inputs and expected outputs for those tests are provided in the `.in/.out` files in `cscb07f26/a2`.

Submission:

Due on Due Date 2: Submit `calc-impl.cc` to MarkUs.

Question 2

2. **(DD1: 15%, DD2: 20%)**In this exercise, you will write a C++ class to control a simple Mars rover exploring the surface of the planet. The terrain is modelled as a map divided into square cells. One step moves the rover to an adjacent cell to the north, east, south, or west. Your rover lands at coordinates (0,0), facing North.

   Some starter code has been provided for you in the `cscb07f26/a2` directory, including the test harness `main.cc`, along with a sample executable. **You may not change the contents of `rover.cc` and `position.cc` (the module interface files) other than by adding private instance members and comments, i.e., the public interface must stay exactly the same.**

   Use the class `Position` declared in `position.cc` for the coordinates.

```
class Position {
  int ew, ns;
  // public methods; see position.cc for details
};
```

   A `Position` is constructed as `Position(ew, ns)`, and both arguments default to 0.

   The east-west direction is the first component of a position (east is positive, west is negative). The north-south direction is the second (north is positive, south is negative).

   The `operator<<` for a `Position` must also be implemented, which should print the current values of the position in the format: `(ew,ns)` (no spaces or line breaks). Please ensure that your output matches that of the sample executable.

   Use the class `Rover` declared in `rover.cc` to implement the rover's functionality.

```
class Rover {
  Position cur; // current position
  int dir;  // 0 = N, 1 = E, 2 = S, 3 = W
  int numSteps; // total number of steps travelled by the rover
  // public methods; see rover.cc for details
};
```

   The `operator<<` for a `Rover` must also be implemented, which should print the current status of the rover as: `Current position: (ew,ns), Facing <direction>`, where `<direction>` is `North`, `East`, `South`, or `West` (no line breaks). Please ensure that your output matches that of the sample executable.

   You must create the files `position-impl.cc` and `rover-impl.cc` and implement all the operations specified in `position.cc` and `rover.cc`.

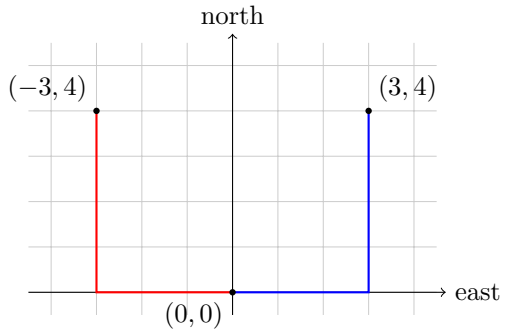
   The test harness `main.cc` is provided, with which you may interact with your rover for testing purposes. **The test harness is not robust, and you are not to devise tests for it, just for the `Rover` class. Do not change this file.**

   The test harness supports the following commands, which can be thought of as commands sent to the rover from Earth:

Command	Description
`f` n	Moves forward by n steps in the direction the rover is facing. n must be positive.
`b` n	Moves backward by n steps, i.e., opposite to the direction the rover is facing. n must be positive.
`l`	Turns 90 degrees to the left (counterclockwise).
`r`	Turns 90 degrees to the right (clockwise).
`c`	Prints the current status of the rover.
`t`	Prints the total number of steps travelled by the rover.
`m`	Prints the Manhattan distance of the current position to the origin.

   Notes:

- The total number of steps travelled by the rover (its "odometer reading", returned by `totalDistance()`) is a simple sum of all the forward and backward steps. For example, if the rover travels 5 steps forward, 2 backward, turns right, and travels 5 steps forward, the total number of steps is just $5 + 2 + 5 = 12$.
- Moving backward does not change the direction the rover is facing. For example, if the rover is at `(0,0)` facing North, then `b 2` moves it to `(0,-2)`, and it is still facing North.
- Turning changes only the direction the rover is facing; it does not change the position or the total number of steps.
- The default for the number of steps in `forward` and `backward` is 1, so calling `forward()` moves the rover one step. The behaviour of these methods is not specified when the number of steps is not positive, and it will not be tested.
- The Manhattan distance is calculated as the absolute north-south displacement plus the absolute east-west displacement. For example, the Manhattan distance of the position `(3,4)` to the origin is 7 (i.e., $3 + 4$). The Manhattan distance of the position `(-3,4)` to the origin is also 7. See diagram below.



A sample session follows (lines in black are input, lines in blue are output):

```
c
Current position: (0,0), Facing North
f 5
c
Current position: (0,5), Facing North
r
f 3
b 1
c
Current position: (2,5), Facing East
t
9
m
7
l
c
Current position: (2,5), Facing North
```

Submission:

(a) **Due on Due Date 1:** List the name of your test in your suite file `suiteq2.txt`. Zip your suite file, together with the associated `.in` and `.out` files, into the file `a2q2a.zip`.

(b) **Due on Due Date 2:** Implement this program in C++ and place the files `rover.cc`, `rover-impl.cc`, `position.cc`, and `position-impl.cc` in the zip file, `a2q2b.zip`.

Question 3

3. **(DD1: 25%, DD2: 30%)**In this exercise, you will write a class that manages an ASCII canvas: a rectangular grid of characters ("pixels") that can be drawn on and printed.

   For example, this is how a canvas with 6 columns and 3 rows is printed (some of the pixels have been changed from the initial dot):

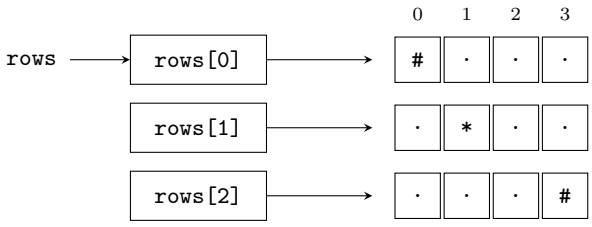
```
+------+
|#.....|
|..**..|
|.....#|
+------+
```

   The pixel in the top-left corner has coordinates $(0, 0)$. The first coordinate, x, is the column and the second coordinate, y, is the row. So, in the canvas above, the pixel at $(0, 0)$ is `#`, the pixel at $(2, 1)$ is `*`, and the pixel at $(5, 2)$ is `#`.

   We have provided starter code in the `cscb07f26/a2` directory: the declaration of the `Canvas` class in the module interface file `canvas.cc`, and the test harness described below (`main.cc`, `controller.cc`, and `controller-impl.cc`). **You may not change the contents of `canvas.cc` other than by adding private members and comments, i.e., the public interface must stay exactly the same.**

```
class Canvas {
    int width;    // number of columns (0 if the canvas is empty)
    int height;   // number of rows (0 if the canvas is empty)
    char **rows;  // array of height pointers; rows[y] points to an array of width
                  // characters (nullptr if the canvas is empty)
    // public methods; please check canvas.cc for details
};
```

   The diagram below shows how a canvas is stored in memory.



   Your job is to implement all the methods of the `Canvas` class in the file `canvas-impl.cc`. Please read the documentation for each method in `canvas.cc` carefully. The comments explain what they must do.

   Implementation details:

   - A canvas is *empty* if it has no pixels: its `width` and `height` are 0 and `rows` is `nullptr`. A canvas is empty only after its contents have been moved to another canvas.
   - A canvas created with `Canvas(int width, int height, char ch)` has the given number of columns and rows (which are always positive), and every one of its pixels is `ch` (by default, a dot, `'.'`). A canvas that is created as a copy of another canvas, or that is assigned a copy of another canvas, must end up with the same dimensions and pixels as the original, and from then on the two canvases must be completely independent: changing one of them never changes the other. This must work no matter what the dimensions of the two canvases are, and whether either of them is empty (a copy of an empty canvas

is empty). When the contents of a canvas are moved to another canvas (either by constructing it or by assigning to it), the target ends up with exactly what the original had, without any pixel being copied, and the original is left empty. Assigning a canvas to itself, whether by copying or by moving, must leave it unchanged.

- The `print` method prints a non-empty canvas as a frame around its rows of pixels, as shown in the example above: a line with `+`, *width* dashes, and `+`; then one line for each row, with `|`, the pixels of the row, and `|`; and, finally, the same line of dashes again. Each line ends with a newline. An empty canvas is printed as a single line containing `(empty)`.
- Because we test your submission by comparing its output with the expected output of the correct program, you will lose marks if your output does not precisely match the output of our implementation. We are also providing you with a sample executable that produces the correct output. Please test the provided executable for every potential type of output and ensure that your solution matches the sample output precisely (use a utility like `diff` to ensure that the outputs are identical; don't rely only on a visual inspection). The `produceOutputs` and `runSuite` scripts can help you do that.
- **Your program must not leak any memory.**

A test harness is provided in `main.cc` (which uses the module `controller`, provided in the files `controller.cc` and `controller-impl.cc`). You can use it to test your `Canvas` class. **The test harness is not robust, and you are not to devise tests for it, just for the `Canvas` class. Do not change these files.** We have also provided a sample executable (`canvas`) that you can use for tests.

The test harness keeps track of up to four canvases, called `A`, `B`, `C`, and `D`. Initially, none of them exist. The commands below create canvases, copy and move them, and so on. The arguments X and Y are canvas names (a single capital letter from `A` to `D`), w, h, x, and y are integers, and c is a single character that is not whitespace. Please read the contents of `main.cc`, `controller.cc`, and `controller-impl.cc` if you need more details. The `Controller` class owns the canvases, and it creates, copies, moves, and deletes them as requested by the commands.

The harness prints nothing when it starts and does not print a prompt. Messages listed below as "error" are printed to the standard error stream (`cerr`) and the command has no other effect; all the other messages are printed to the standard output. Please ensure that both your standard output and standard error match those of the sample executable. (In the `.out` files of the tests, the two streams appear interleaved in the order in which they were printed.) The program ends when the command `quit` is read or when there is no more input.

Every command that takes a canvas name as an argument first checks the name. If it is not one of `A`, `B`, `C`, or `D`, the error is `Invalid canvas name.` For all the commands except `new`, and for the first argument of `copy` and `move`, if the canvas does not exist, the error is `Canvas` X `does not exist.`

Command	Description	Output
`new` X w h c	Creates the canvas X with w columns and h rows, with every pixel set to c.	`Canvas` X `created (`w`x`h`).` Errors: `Canvas` X `already exists.` and, if w or h is not positive, `Width and height must be positive.`
`copy` X Y	Makes canvas Y a copy of canvas X. If Y does not exist, it is created as a copy of X. If Y already exists, its contents are replaced by a copy of the contents of X. X and Y may be the same canvas.	`Canvas` Y `is now a copy of canvas` X`.`

Command	Description	Output
`move` *X Y*	Moves the contents of canvas *X* to canvas *Y*. If *Y* does not exist, it is created with the contents of *X*. If *Y* already exists, its contents are replaced by the contents of *X*. Afterwards, *X* still exists but it is empty (unless *X* and *Y* are the same canvas, which is left unchanged).	`Canvas` *X* `was moved to canvas` *Y*.
`set` *X x y c*	Sets the pixel at column *x* and row *y* of canvas *X* to *c*.	`Set (`*x*`,`*y*`) of canvas` *X* `to '`*c*`'.` Error if the position is not inside the canvas (or if the canvas is empty): `Position (`*x*`,`*y*`) is outside canvas` *X*.
`get` *X x y*	Gets the pixel at column *x* and row *y* of canvas *X*.	`(`*x*`,`*y*`) of canvas` *X* `is '`*c*`'.`, where *c* is the pixel. Error: same as `set`.
`fill` *X c*	Sets every pixel of canvas *X* to *c*.	`Canvas` *X* `filled with '`*c*`'.`
`print` *X*	Prints canvas *X*.	The canvas, as described above.
`size` *X*	Prints the dimensions of canvas *X*.	`Canvas` *X* `is` *w*`x`*h*`.` or, if the canvas is empty, `Canvas` *X* `is empty.`
`delete` *X*	Deletes canvas *X*. Afterwards, the canvas does not exist (and can be created again).	`Canvas` *X* `deleted.`
`quit`	Quits the program.	None.

A sample session follows (lines in black are input, lines in blue are output):

```
new A 4 3 .
Canvas A created (4x3).
set A 0 0 #
Set (0,0) of canvas A to '#'.
set A 3 2 #
Set (3,2) of canvas A to '#'.
print A
+----+
|#...|
|....|
|...#|
+----+
copy A B
Canvas B is now a copy of canvas A.
set B 1 1 *
Set (1,1) of canvas B to '*'.
print A
+----+
|#...|
|....|
|...#|
+----+
print B
+----+
|#...|
|.*..|
|...#|
+----+
```

```
move B C
Canvas B was moved to canvas C.
print B
(empty)
print C
+----+
|#...|
|.*..|
|...#|
+----+
size B
Canvas B is empty.
size C
Canvas C is 4x3.
```

Note that, in the command `copy A B`, canvas `B` did not exist, so it was created using the copy command. Also note that, after `B` was moved to `C`, canvas `B` still exists, but it is empty.

Submission:

(a) **Due on Due Date 1:** List the name of your test in your suite file `suiteq3.txt`. Zip your suite file, together with the associated `.in` and `.out` files, into the file `a2q3a.zip`.

(b) **Due on Due Date 2:** Implement this in C++ and place the files `canvas.cc` and `canvas-impl.cc` in the zip file, `a2q3b.zip`. We will provide the `main.cc`, `controller.cc`, and `controller-impl.cc` to test your submission.

Submission

The following files are due at Due Date 1: `a2q2a.zip`, `a2q3a.zip`.
The following files are due at Due Date 2: `calc-impl.cc`, `a2q2b.cc`, `a2q3b.zip`.